```
def ackley(x):
    # a modification of https://www.sfu.ca/~ssurjano/ackley.html
    return -20 * torch.exp(
        -0.2 * torch.sqrt((x[:, 0] ** 2 + x[:, 1] ** 2) / 2)
    )
    <linearrow />    - torch.exp(torch.cos(2 * pi * x[:, 0] / 3)
    <linearrow />    + torch.cos(2 * pi * x[:, 1]))
```

We specifically restrict the domain of this function to the square region that is between -3 and 3 in both dimensions, which is typically denoted as `[-3, 3]`^{2}. To visualize this objective function, we use the heat map in Figure 3.16.

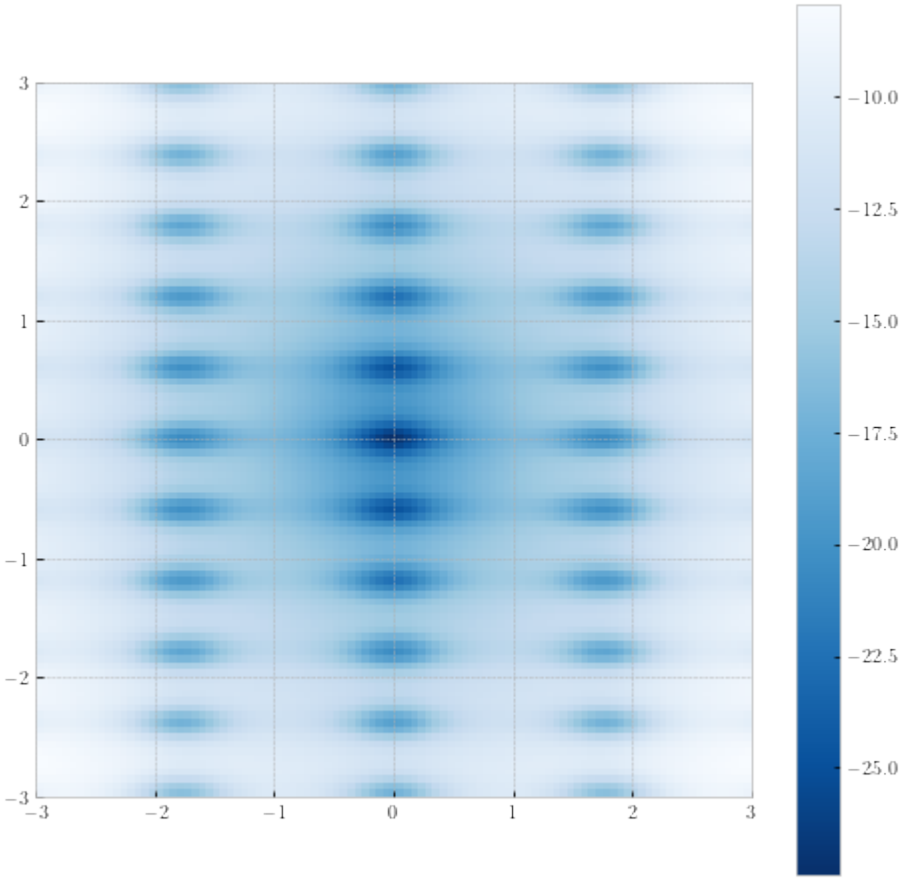


Figure 3.16 The two-dimensional Ackley function to be used as our objective. Here, the x-axis has less variability (it changes less) than the y-axis, requiring different length scales.

Each dark blob in the heat map can be thought of as a "valley" that has a low value across the surface of the objective function. Here, there are many more valleys across the *y*-axis than across the *x*-axis, indicating that the second dimension has more variability than the first.

Once again, this means using only one length scale for both dimensions is not a good choice. What we should do instead is to have a length scale for *each* dimension (two in this case). Each length scale can then be independently optimized using gradient descent.

©Manning Publications Co. To comment go to liveBook
https://livebook.manning.com/#!/book/bayesian-optimization-in-action/discussion

Licensed to ashwin sampath <ashwin.manning@gmail.com>